
# DESIGN AND ANALYSIS OF ALGORITHMS

# DESIGN AND ANALYSIS OF ALGORITHMS

## Decrease by Variable size Algorithm

The third principal variety of decrease-and-conquer, the **size reduction pattern** varies from one iteration of the algorithm to another.

This algorithm helps optimize:

1. Computing a Median and the Selection Problem
2. Interpolation Search
3. Searching and Insertion in a Binary Search Tree
4. The Game of Nim

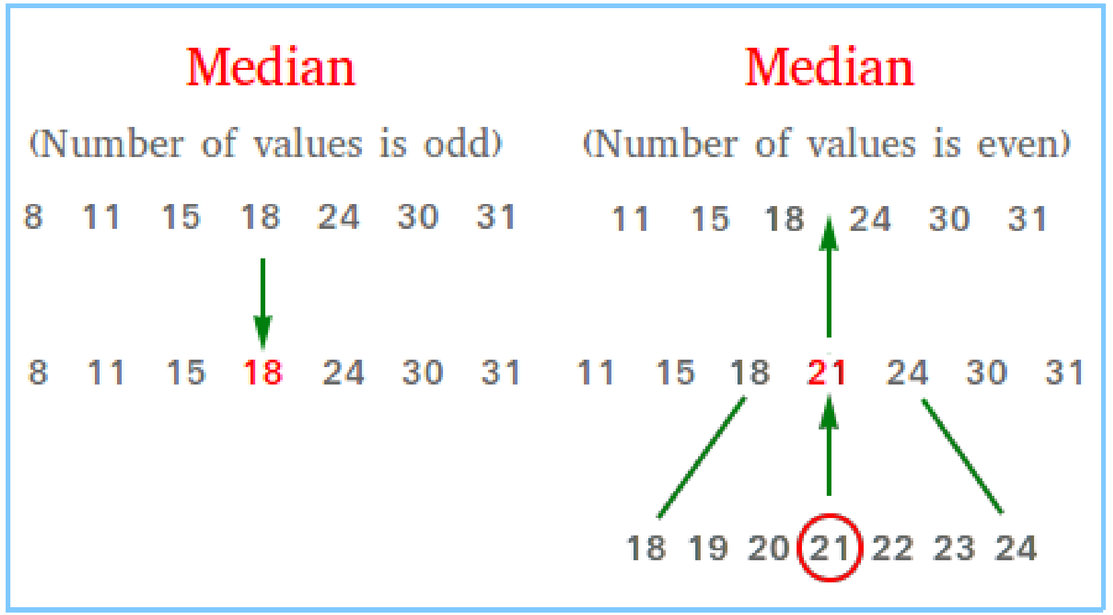
We will look at **Computing a Median and the Selection Problem** in detail.

# Computing a Median and the Selection Problem

**Objective:** Find the median of an odd array in the best time.

## What is the Median?

The median is the middle value of a sorted array.




## Naïve Approach:

1. Sort the array.
2. Find the element in the middle position.

## Time complexity:

- Sorting: O(n log n)
- Selecting the middle element: O(1)
- Total: O(n log n)

## Computing a Median and the Selection Problem

### Issues with the Naïve Approach:

- Sorting places all elements in their correct positions.
- We only need the middle element, so sorting the entire array is unnecessary.

### What can be optimized:

- Instead of sorting all elements, directly try to find the element at the n/2 position.
- This problem of finding the k-th smallest or largest element is called the **Selection Problem**

### Proposed Solution:

- QuickSelect to solve the Selection Problem

## Computing a Median and the Selection Problem

### Quick Select:

- Similar to Quick Sort, we pick a **pivot** element and **partition** the array:
  - Elements **smaller** than the pivot go to the **left**.
  - Elements **larger** than the pivot go to the **right**.
  - The **pivot** is placed in its **correct** position.
  - This partition technique is called **Lomuto Partition**.

- Difference from Quick Sort:
  - Quick Sort recursively sorts the entire array.
  - Quick Select **stops** once the pivot is at the k-th position (n/2 for median).

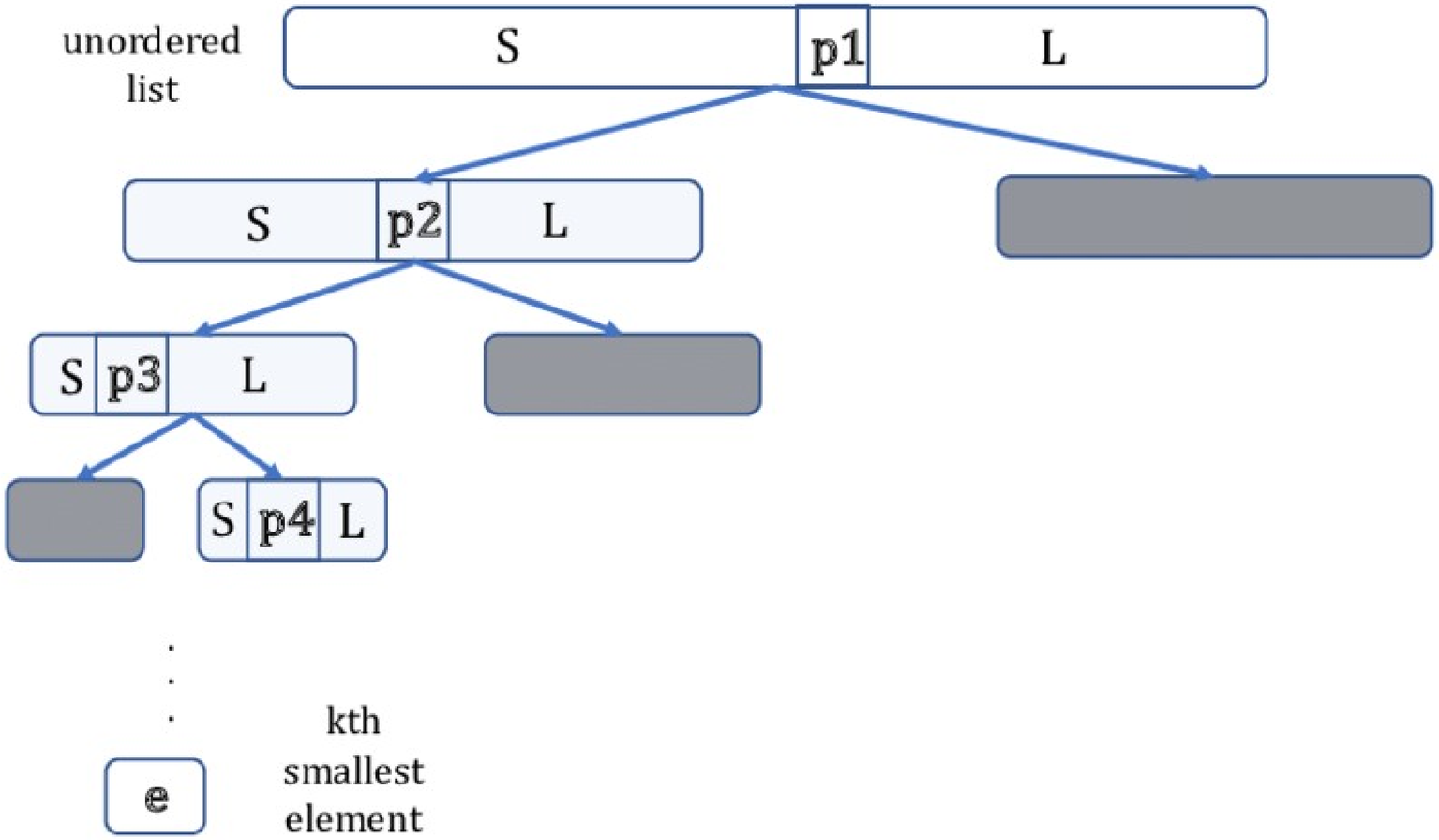
## Algorithm for QuickSelect

**Input : List, left is first position of list, right is last position of list and k is k-th smallest element.**
**Output : A new list is partitioned.**

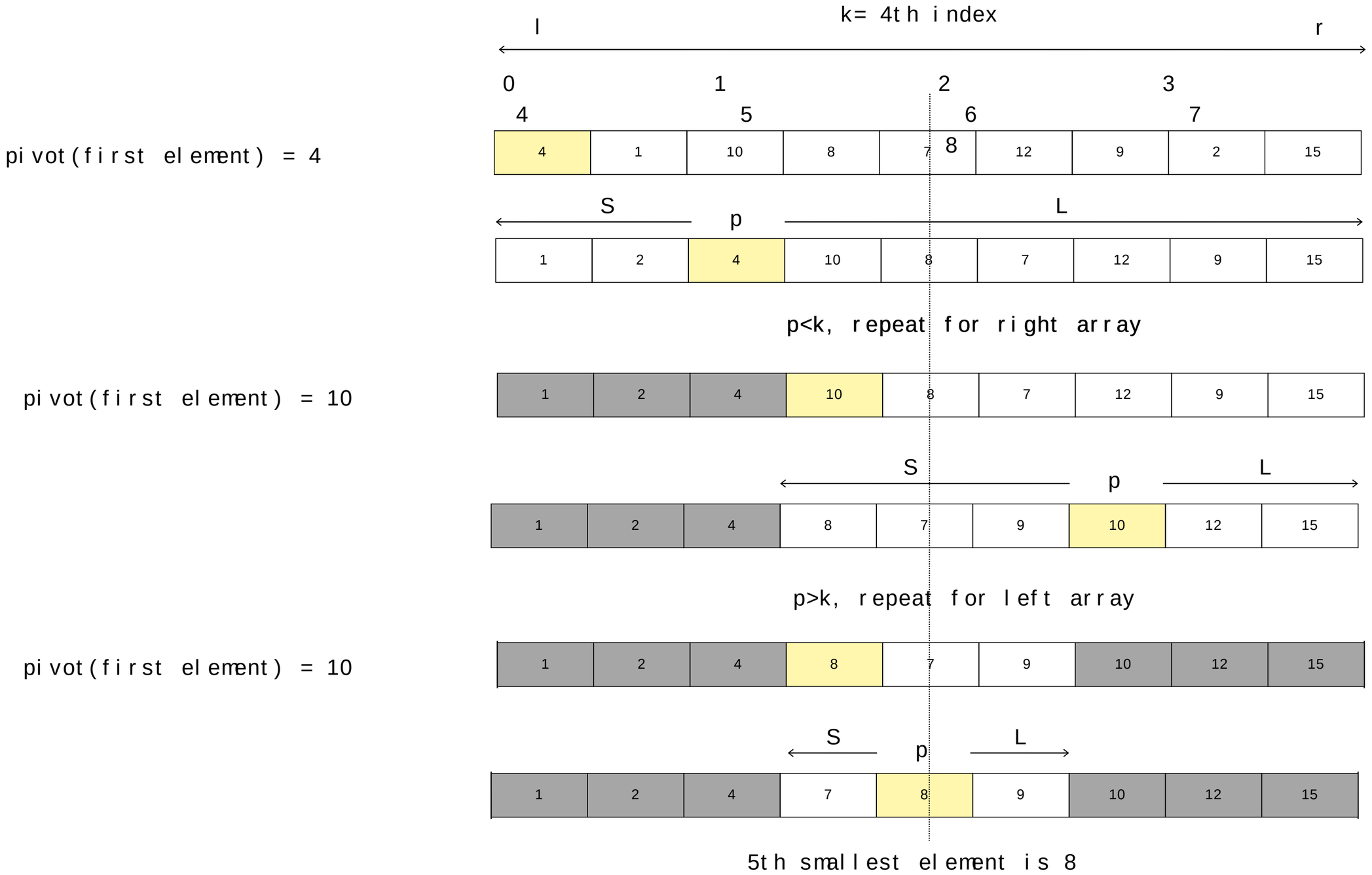
```
quickSelect(list, left, right, k)
  if left = right
    return list[left]

  // Select a pivotIndex between left and right
  pivotIndex <= partition(list, left, right, pivotIndex)
  if k = pivotIndex
    return list[k]
  else if k < pivotIndex
    right <= pivotIndex - 1
  else
    left <= pivotIndex + 1
```

# QuickSelect



# Find the 5th smallest element in the given array



# Time Complexities for QuickSelect

## Best Case:

- The best case occurs when the pivot chosen is always the median of the array. This perfectly divides the array in half at each step.
- The recurrence relation: $T(n)=T(n/2)+O(n)$
- Solving this gives $O(n)$ time complexity, and the problem size is reduced exponentially.

## Worst Case:

- The worst case occurs when the pivot selection is poor, e.g., always choosing the smallest or largest element. This results in highly unbalanced partitions, where one side has n - 1 elements and the other has 0.
- The recurrence becomes: $T(n)=T(n-1)+O(n)$
- Solving this gives $O(n^2)$ time complexity.

## Improve the Worst Case Complexity

There are two key ways to achieve better worst-case complexity:

1. Choosing a Better Pivot Selection Strategy such as Median of Medians Algorithm.
2. Using an Improved Partitioning Algorithm such as using Hoare's Partition Scheme instead of Lomuto's Partition Scheme can improve efficiency by reducing swaps.

**Median of Medians Algorithm**(Guaranteed O(n) Worst-Case):

We need to **choose a pivot close to the median** consistently. This ensures better partitioning and prevents highly unbalanced cases.

- Divide the array into groups of 5 elements (or another small fixed size).
- Find the median of each group.
- Recursively find the median of these medians to use as the pivot.
- Ensures a pivot that is reasonably close to the true median, avoiding worst-case $O(n^2)$.
- Guarantees O(n) worst-case time complexity.

## Variable Decrease and Conquer

By utilizing **variable-size Decrease and Conquer**, we optimized the Selection Algorithm, making median computation more efficient.

Unlike traditional divide-and-conquer, where the problem size is reduced by a fixed factor (e.g., halving in binary search), **variable-size** reduction allows for **adaptive problem**shrinking at each step. In Quick Select, this means:

- Instead of processing the entire array, we eliminate a portion based on the pivot's position.
- The size reduction varies **dynamically**, depending on where the pivot is placed.
- This flexibility enables an **O(n)** average-time complexity, significantly improving over naive sorting methods.

By using variable Decrease and Conquer, we were able to directly target the element we needed, making the solution far more efficient.

# THANK YOU